

Python: exercises on math module

This notebook was generated in **student** mode.

Exercise 1: Hypotenuse Calculator

Exercise Description

Write a function `calculate_hypotenuse(a, b)` that takes two sides of a right triangle and calculates the length of the hypotenuse using `math.hypot()`.

The function should return the hypotenuse length as a float.

Your solution

```
import math

# Write your function here
def calculate_hypotenuse(a, b):
    pass
```

Test your solution

```
# Test the function with known right triangle values
assert calculate_hypotenuse(3, 4) == 5.0 # 3-4-5 triangle
assert calculate_hypotenuse(5, 12) == 13.0 # 5-12-13 triangle
assert calculate_hypotenuse(8, 15) == 17.0 # 8-15-17 triangle
```

Test your solution

```
# Test with decimal values
import math
assert abs(calculate_hypotenuse(1, 1) - math.sqrt(2)) < 0.0001
assert calculate_hypotenuse(0, 5) == 5.0 # One side is zero
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML; import urllib.parse as u; HTML('<a href="https://pyth
```

Exercise 2: Sine Angle Calculator

Exercise Description

Create a function `sine_of_angle(degrees)` that takes an angle in degrees, converts it to radians using `math.radians()`, and then calculates the sine of the angle using `math.sin()`.

The function should return the sine value as a float.

Your solution

```
import math

# Write your function here
def sine_of_angle(degrees):
    pass
```

Test your solution

```
# Test the function with known angle values
import math
assert abs(sine_of_angle(0) - 0) < 0.0001 # sin(0°) = 0
assert abs(sine_of_angle(30) - 0.5) < 0.0001 # sin(30°) = 0.5
assert abs(sine_of_angle(90) - 1) < 0.0001 # sin(90°) = 1
```

Test your solution

```
# Test with 45 degrees (should be approximately sqrt(2)/2)
import math
expected_45 = math.sqrt(2) / 2
assert abs(sine_of_angle(45) - expected_45) < 0.0001
assert abs(sine_of_angle(180) - 0) < 0.0001 # sin(180°) = 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Compound Interest Calculator

Exercise Description

Write a function `compound_interest(principal, rate, times_compounded, years)` that calculates the compound interest using the formula:

$$A = P \times (1 + r/n)^{(nt)}$$

Where:

- A = the final amount after interest
- P = the principal amount (initial deposit)
- r = the annual interest rate (as a decimal)
- n = the number of times the interest is compounded per year
- t = the number of years

The function should take as input the principal amount, annual interest rate, number of times the interest is compounded per year, and the number of years. Use `math.pow()` for exponentiation.

The function should return the final amount as a float.

Your solution

```
import math
```

```
# Write your function here
def compound_interest(principal, rate, times_compounded, years):
    pass
```

Test your solution

```
# Test the function with known values
# $1000 at 5% compounded quarterly for 10 years should be approximately $1643.62
result = compound_interest(1000, 0.05, 4, 10)
assert abs(result - 1643.62) < 0.1

# Test with simple case: $100 at 10% compounded annually for 1 year should be $110
result = compound_interest(100, 0.10, 1, 1)
assert abs(result - 110.0) < 0.01
```

Test your solution

```
# Test edge cases
# Zero interest rate should return the principal
result = compound_interest(1000, 0.0, 4, 10)
assert result == 1000.0

# Zero years should return the principal
result = compound_interest(1000, 0.05, 4, 0)
assert result == 1000.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML; import urllib.parse as u; HTML('<a href="https://pyth
```

